

Table des matières

Labo 07 — Le même travail, deux moteurs	1
Avant de commencer — l'image de Spark.....	2
Étape 1 — Demander tout l'historien, et refaire le bronze sur une machine	2
Étape 2 — Les agrégats sur une machine, et le plan qui les explique	3
Étape 3 — Le même bronze, en PySpark, dans un conteneur à usage unique	4
Étape 4 — Comparer ce que les deux moteurs ont écrit.....	7
Étape 5 — Les agrégats en Spark, le brassage dans le plan, et la comparaison ligne à ligne	8
Étape 6 — Dédoublonner la télémétrie, le pire cas pour Spark	11
Étape 7 — Les trois pièges, mesurés.....	12
Étape 8 — Conclure, remettre le projet d'aplomb, versionner	14
Récapitulatif	16
Dépannage	16
Annexe — Sur Databricks.....	17

Labo 07 — Le même travail, deux moteurs

Labo · Module 7/12 — Formation Data Engineering

Ce que vous allez faire. Demander à l'usine tout son historien, deux fois plus de points qu'au Cours 06, et refaire le bronze avec le pipeline que vous avez écrit. Écrire ensuite le même traitement en PySpark, le lancer dans un conteneur à usage unique, et comparer : les résultats, les durées, les fichiers écrits et les deux plans d'exécution. Provoquer les trois pièges classiques et les mesurer. Objectif de fin : les mêmes 47 618 agrégats des deux côtés, un tableau de durées qui tient en cinq lignes, et une décision défendable sur le moteur qu'il faut à Boréal.

Comptez une séance d'environ deux heures, dont une bonne vingtaine de minutes d'attente : deux générations de données et quatre lancements de Spark. Qui le fait en deux fois s'arrête après l'étape 5, quand la comparaison est faite.

Trois terminaux servent dans ce labo. L'Anaconda Prompt, environnement *formations* actif, pour Python, Git et le générateur. La ligne de commande DuckDB pour le SQL de vérification. Ubuntu, dans WSL 2, pour Docker et donc pour Spark : c'est là que vit Docker Engine, et nulle part ailleurs. Chaque étape dit dans lequel on se trouve.

La liste de contrôle du cours :

- ☐ L'image de Spark tirée : `docker images` montre `spark:4.2.0-scala2.13-java21-python3-ubuntu`, 2,22 Go
 - ☐ Les sources régénérées avec l'historien complet : 2 628 fichiers, 269,9 Mo, empreinte `0e53ccf4c7b8e73b`, historien de 821 jours et 2 407 796 lignes
 - ☐ Le bronze refait sur une machine : `python -m pipeline bronze` rend 2 612 281 lignes en une trentaine de secondes, source et bronze d'accord des deux côtés
 - ☐ Les agrégats sur une machine : 47 618 lignes, 2 405 366 points, en moins d'une seconde ; le plan montre `PARQUET_SCAN` et son filtre poussé
 - ☐ Le bronze écrit par Spark : 27 partitions de lecture, 2 407 796 lignes, 821 dossiers de jour
 - ☐ Les deux Parquet comparés : mêmes lignes, 14 Mo en `zstd` contre 24 Mo en `snappy`
 - ☐ Les agrégats en Spark : 47 618 lignes et 2 405 366 points comme sur une machine, 6 moyennes qui diffèrent d'un millième, et un Exchange dans le plan
 - ☐ Les trois pièges mesurés : 18 s contre 8,8 s pour le brassage de trop, 247 879 lignes contre 1 833 dans les partitions, et les 2 407 796 lignes ramenées au pilote
 - ☐ Le projet remis dans l'état du Cours 06, le dossier `spark/` et le `.gitattributes` versionnés
-

Avant de commencer — l'image de Spark

Ce cours emploie un seul outil neuf, et il ne s'installe pas : il se tire une fois, comme image de conteneur, et il ne reste rien entre deux lancements. Docker Engine, lui, tourne déjà dans l'Ubuntu de WSL 2 depuis le Cours 04 ; `Data_Domain/GUIDE_INSTALLATION.md` le monte, étape 5.

Dans l'Anaconda Prompt, depuis `Data_Domain\Projet_Boreal` :

```
ws1
```

Vous voilà dans Ubuntu, au même endroit. Tirez l'image officielle, 2,22 Go, une fois pour toutes :

```
docker pull spark:4.2.0-scala2.13-java21-python3-ubuntu
docker images spark
```

L'image porte Spark 4.2.0, Python et Java 21. Aucun service ne démarre : une image n'est pas un conteneur. Si la pile Airflow du Cours 06 tourne encore, laissez-la, elle ne gêne pas ; le conteneur de ce cours ne publie aucun port et disparaît à la fin de chaque lancement.

Étape 1 — Demander tout l'historien, et refaire le bronze sur une machine

Théorie : `Theorie_07.md`, section 1.

Jusqu'ici, l'usine ne livrait que trois mois d'historien, plus les mois de 2026 ajoutés au Cours 06. Pour ce cours, elle ouvre ses archives depuis la mise en service du SCADA, le 2024-01-01. Dans l'Anaconda Prompt, depuis Data_Domain, et non depuis Projet_Boreal :

```
python _univers/boreal/generer_boreal.py --projet Projet_Boreal --jusqua 2026-03 --historien complet
```

Deux options, et elles se cumulent. --jusqua 2026-03 garde les mois de 2026 que le Cours 06 a fait arriver ; --historien complet remplace les 182 jours d'historien par les 821 jours livrables. La génération prend environ deux minutes et demie et refait le dossier sources/ en entier.

```
usine/      historien 821 jours, 2 407 796 lignes, 58 étiquettes ; 825 bons de travail ; 148 891 relevés du compteur
telemetrie/ 974 jours, 204 485 messages reçus de 7 644 véhicules, dont 2 943 doublons
```

```
2 628 fichiers, 269,9 Mo, empreinte globale 0e53ccf4c7b8e73b...
Durée : 150.8 s.
```

L'historien pèse maintenant 117 Mo en CSV, contre 8 Mo au Cours 06. C'est le jeu de données de tout le labo.

Ce qui dérive de ces fichiers ne s'est pas mis à jour tout seul : le bronze du Cours 02 contient encore les 550 087 lignes d'hier. Refaites-le avec le pipeline du Cours 05, dans l'Anaconda Prompt, depuis 01_de :

```
python -m pipeline bronze
```

```
INFO pipeline :   historien      2 407 796 lignes, source et bronze d'accord
INFO pipeline :   télémétrie     204 485 lignes, source et bronze d'accord
INFO pipeline :   étape bronze (...) : fin, 2 612 281 lignes en 29.3 s
```

Trente secondes, contrôle compris, pour 2,6 millions de lignes. Le contrôle est la moitié du temps : il recompte les CSV et les Parquet, des deux côtés, et c'est lui qui vous autorise à croire le résultat.

Pour chronométrer la conversion seule, sans son contrôle, lancez le script du Cours 02 dans la ligne de commande DuckDB, depuis Projet_Boreal :

```
duckdb -cmd ".timer on" -f 01_de\sql\bronze_historien.sql
```

Attendu : le COPY rend Run Time (s): real 2.6 chez l'auteur, puis le contrôle rend 2 407 796 des deux côtés en une vingtaine de secondes. Retenez ce 2,6 : c'est la barre que Spark va devoir franchir.

Piège — un chronométrage qui mesure le lot entier. duckdb -c "requête1; requête2" chronomètre les deux ensemble. Une requête à chronométrer se lance seule, dans sa propre invocation. C'est pour la même raison que le script et le contrôle se mesurent à part.

Étape 2 — Les agrégats sur une machine, et le plan qui les explique

Théorie : Theorie_07.md, sections 1 et 5.

Le traitement qui servira de mesure étalon : par étiquette et par jour, le nombre de points de qualité bonne, leur moyenne, leur minimum et leur maximum. Dans la ligne de commande DuckDB, depuis `Projet_Boreal`. Créez d'abord le dossier de comparaison, dans l'Anaconda Prompt :

```
mkdir comparaison\duckdb
```

Puis, dans la ligne de commande DuckDB :

```
duckdb -cmd ".timer on" -c "COPY (SELECT TagName AS etiquette, CAST(Timestamp AS DATE) AS jour, COUNT(*) AS points,
ROUND(AVG(Value), 3) AS moyenne, MIN(Value) AS minimum, MAX(Value) AS maximum FROM
'plateforme/lac/bronze/usine/historien/**/*.parquet' WHERE Quality = 192 GROUP BY TagName, CAST(Timestamp AS DATE)) TO
'comparaison/duckdb/agregats.parquet' (FORMAT parquet);"
```

Attendu : Run Time (s): real 0.17 chez l'auteur. Le contrôle, à lancer ensuite :

```
-- Ce que Le moteur d'une machine a produit.
-- Attendu : 47 618 agrégats, 2 405 366 points, somme des moyennes 2 737 589,987.
SELECT COUNT(*) AS agregats, SUM(points) AS points, ROUND(SUM(moyenne), 3) AS somme_moyennes
FROM 'comparaison/duckdb/agregats.parquet';
```

Deux dixièmes de seconde pour parcourir 2,4 millions de points et en rendre 47 618. Le plan dit pourquoi :

```
EXPLAIN SELECT TagName, CAST(Timestamp AS DATE) AS jour, COUNT(*) AS points
FROM 'plateforme/lac/bronze/usine/historien/**/*.parquet' WHERE Quality = 192
GROUP BY TagName, CAST(Timestamp AS DATE);
```

Le plan se lit de bas en haut, et son dernier bloc porte l'essentiel :

```
PARQUET_SCAN
Projections:
  TagName
  Timestamp
  Value
Filters: Quality=192
~254,674 rows
```

Trois choses à y voir. Le moteur ne lit que trois colonnes sur cinq, parce que le Parquet est rangé par colonnes. Le filtre sur la qualité descend jusqu'au fichier, mention `Filters` : les lignes écartées ne remontent jamais. Et l'estimation de lignes, ~254,674, n'est qu'une estimation, tirée des statistiques du Parquet. Au-dessus, un `HASH_GROUP_BY` fait tout le regroupement, et rien ne se déplace : un seul processus, une seule mémoire.

Étape 3 — Le même bronze, en PySpark, dans un conteneur à usage unique

Théorie : Theorie_07.md, sections 2 et 3.

Dans l'Anaconda Prompt, depuis 01_de. Le code de Spark vit dans le dépôt, à côté du paquet pipeline, dans un dossier à lui :

```
mkdir spark
notepad spark\spark.sh
```

Le lanceur, d'abord. Il tient en une commande, et c'est lui qui décide de tout ce qui entoure Spark :

```
#!/bin/bash
# spark.sh : Lance un script PySpark dans un conteneur à usage unique (Cours 07).
#
# Ce que fait chaque option :
# --rm          Le conteneur disparaît à la fin ; rien ne tourne entre deux lancements
# --user root   Spark change les droits des dossiers qu'il crée, ce qu'un disque Windows monté
#              refuse à tout autre utilisateur
# -v "$PWD"     Le projet entre dans le conteneur, à l'endroit que -w rend courant
# local[*]      un pilote et des exécuteurs dans le même processus, sur tous les cœurs
set -e
docker run --rm \
  --user root \
  -v "$PWD":/projet \
  -w /projet \
  spark:4.2.0-scala2.13-java21-python3-ubuntu \
  /opt/spark/bin/spark-submit --master "local[*]" "$1"
```

Piège — le fichier enregistré en .txt, encore. Le Bloc-notes ajoute l'extension quand on ne choisit pas « Tous les fichiers ». Vérifiez par `dir spark : spark.sh`, et rien d'autre.

Ce fichier-ci sera lu par bash, dans Ubuntu, et le Bloc-notes termine ses lignes à la mode de Windows, par un retour chariot suivi d'un saut de ligne. Bash prend ce retour chariot pour un caractère de la commande. Retirez-les, dans Ubuntu, depuis `Projet_Boreal` :

```
sed -i 's/\r$//' 01_de/spark/spark.sh
```

Piège — `set: -: invalid option`, puis `docker: invalid reference format`. C'est le retour chariot de Windows, resté en fin de chaque ligne du script. Le message ne le nomme jamais : bash lit `set -e\r` et cherche une option `-\r`. La commande `sed` ci-dessus le règle, et le geste vaut pour tout script shell écrit sous Windows et lancé sous Linux.

Le premier script, maintenant : la conversion de l'historien, le même travail qu'à l'étape 1.

```
notepad spark\convertir_historien.py

# -*- coding: utf-8 -*-
"""convertir_historien.py : Le bronze de l'historien, écrit par Spark (Cours 07)."""
import time
```

```

from pyspark.sql import SparkSession, functions as F, types as T

SOURCE = "sources/usine/historien/*.csv"
SORTIE = "comparaison/spark/historien"

# Le schéma se déclare, comme au Cours 02 : deviner coûte une lecture de plus et se trompe.
SCHEMA = T.StructType([
    T.StructField("TagName", T.StringType()),
    T.StructField("Timestamp", T.TimestampType()),
    T.StructField("Value", T.DoubleType()),
    T.StructField("Quality", T.IntegerType()),
])

spark = SparkSession.builder.appName("boreal-bronze-historien").getOrCreate()
spark.sparkContext.setLogLevel("WARN")

historien = (spark.read
    .option("header", True)
    .option("timestampFormat", "yyyy-MM-dd HH:mm:ss")
    .schema(SCHEMA)
    .csv(SOURCE)
    # historien_2025-10-28.csv donne la partition jour=2025-10-28
    .withColumn("jour", F.to_date(F.regexp_extract(
        F.input_file_name(), r"historien_(\d{4}-\d{2}-\d{2})", 1))))

print("cours 07 | partitions de lecture :", historien.rdd.getNumPartitions())
print("cours 07 | partitions de brassage :", spark.conf.get("spark.sql.shuffle.partitions"))
print("cours 07 | cœurs vus :", spark.sparkContext.defaultParallelism)

debut = time.perf_counter()
historien.write.mode("overwrite").partitionBy("jour").parquet(SORTIE)
print(f"cours 07 | conversion : {time.perf_counter() - debut:.1f} s")

# Contrôle : autant de lignes écrites que reçues.
# Attendu : 2 407 796, le compte du bilan de la génération.
debut = time.perf_counter()
lignes = spark.read.parquet(SORTIE).count()
print(f"cours 07 | lignes écrites : {lignes} (relues en {time.perf_counter() - debut:.1f} s)")

spark.stop()

```

Rien dans ce fichier ne lit un seul octet avant la ligne `historien.write`. Les trois `print` qui précèdent interrogent le plan et la configuration, pas les données. C'est l'évaluation paresseuse de la section 3 de la théorie, et elle se constate ici : les deux premiers `print` sortent instantanément, le troisième attend l'écriture.

Lancez, dans Ubuntu, depuis `Projet_Boreal` :

```
bash 01_de/spark/spark.sh 01_de/spark/convertir_historien.py

cours 07 | partitions de lecture : 27
cours 07 | partitions de brassage : 200
cours 07 | cœurs vus : 12
cours 07 | conversion : 77.3 s
cours 07 | lignes écrites : 2407796 (relues en 19.7 s)
```

Les 821 fichiers donnent 27 partitions, et non 821 ni 12. Une partition vise 128 Mo, et ouvrir un fichier compte pour 4 Mo dans ce calcul : des fichiers de 145 Ko coûtent donc presque uniquement leur ouverture, et Spark en met une trentaine par partition. Les 200 partitions de brassage, elles, ne serviront qu'à l'étape 5.

Le résultat est juste, au chiffre près. La durée, elle, est deux ordres de grandeur au-dessus des 2,6 secondes de l'étape 1, et le total du lancement dépasse deux minutes, démarrage du conteneur compris. Relancez le même script : la durée varie du simple au double d'un passage à l'autre, entre 52 et 80 secondes chez l'auteur. Un cours honnête cite une fourchette, jamais une durée unique.

Piège — `java.io.FileNotFoundException: File sources/usine/historien/*.csv does not exist`. Ce message apparaît dans le journal alors que tout fonctionne. Spark sonde le chemin littéral avant d'étendre le joker, échoue, puis étend le joker et lit les 821 fichiers. Le compte final le prouve. Ne partez pas en chasse : vérifiez le compte, et passez.

Étape 4 — Comparer ce que les deux moteurs ont écrit

Théorie : `Theorie_07.md`, section 6.

Dans l'Anaconda Prompt, depuis `Projet_Boreal`. Les deux moteurs ont écrit le même bronze, au même grain. Regardez ce que chacun a laissé sur le disque :

```
dir plateforme\lac\bronze\usine\historien\jour=2025-10-28
dir comparaison\spark\historien\jour=2025-10-28
```

D'un côté `data_0.parquet`, 22 277 octets. De l'autre part `00000-<identifiant>.c000.snappy.parquet`, 31 756 octets. Le nom porte le numéro de la partition qui l'a écrit, un identifiant unique de tâche, et le codec. Sur l'ensemble du bronze, 14 Mo contre 24 Mo, pour exactement les mêmes lignes.

L'écart vient du codec par défaut : `zstd` au Cours 02, parce que le script le demande ; `snappy` chez Spark, parce que c'est son défaut. Le contenu se vérifie, dans la ligne de commande DuckDB :

```
-- Les deux bronzes, comptés chacun de son côté.
-- Attendu : 2 407 796 des deux côtés. Un écart : un des deux n'a pas été refait.
SELECT
```

```
(SELECT COUNT(*) FROM 'plateforme/lac/bronze/usine/historien/**/*.parquet') AS une_machine,
(SELECT COUNT(*) FROM 'comparaison/spark/historien/**/*.parquet') AS spark;
```

Un dossier par jour des deux côtés, 821 chacun, et un seul fichier par dossier : chaque jour tient dans une partition, parce que chaque fichier source est un jour. Sur des données brassées, Spark écrirait autant de fichiers par dossier qu'il a de partitions, et c'est là que les lacs mal tenus accumulent des dizaines de milliers de fichiers minuscules.

Étape 5 — Les agrégats en Spark, le brassage dans le plan, et la comparaison ligne à ligne

Théorie : Theorie_07.md, sections 4 et 5.

Dans l'Anaconda Prompt, depuis 01_de :

```
notepad spark\agregats_usine.py

# -*- coding: utf-8 -*-
"""agregats_usine.py : Les agrégats quotidiens par étiquette, en Spark (Cours 07)."""
import time

from pyspark.sql import SparkSession, functions as F

BRONZE = "comparaison/spark/historien"
SORTIE = "comparaison/spark/agregats"

spark = SparkSession.builder.appName("boreal-agregats-usine").getOrCreate()
spark.sparkContext.setLogLevel("WARN")

agregats = (spark.read.parquet(BRONZE)
            .filter(F.col("Quality") == 192)
            .groupBy("TagName", F.to_date("Timestamp").alias("jour"))
            .agg(F.count("*").alias("points"),
                 F.round(F.avg("Value"), 3).alias("moyenne"),
                 F.min("Value").alias("minimum"),
                 F.max("Value").alias("maximum")))

print("cours 07 | le plan, avant de lancer quoi que ce soit :")
agregats.explain(mode="simple")

debut = time.perf_counter()
agregats.write.mode("overwrite").parquet(SORTIE)
print(f"cours 07 | agrégats : {time.perf_counter() - debut:.1f} s")
```

```
# Contrôle : Les mêmes comptes que sur une machine.
# Attendu : 47 618 agrégats et 2 405 366 points. Un écart : Le bronze n'a pas été refait.
controle = (spark.read.parquet(SORTIE)
            .agg(F.count("*").alias("agregats"),
                F.sum("points").alias("points"),
                F.round(F.sum("moyenne"), 3).alias("somme_moyennes"))
            .collect()[0])
print(f"cours 07 | agrégats : {controle['agregats']}, points : {controle['points']}, "
      f"somme des moyennes : {controle['somme_moyennes']}")

spark.stop()
```

Dans Ubuntu, depuis Projet_Boreal :

```
bash 01_de/spark/spark.sh 01_de/spark/agregats_usine.py
```

Le plan sort avant que rien ne soit calculé :

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[TagName, jour], functions=[count(1), avg(Value), min(Value), max(Value)])
   +- Exchange hashpartitioning(TagName, jour, 200), ENSURE_REQUIREMENTS
      +- HashAggregate(keys=[TagName, jour], functions=[partial_count(1), partial_avg(Value), ...])
         +- Project [TagName, Value, cast(Timestamp as date) AS jour]
            +- Filter (isnotnull(Quality) AND (Quality = 192))
               +- FileScan parquet [...] PushedFilters: [IsNotNull(Quality), EqualTo(Quality,192)]
```

Lisez-le de bas en haut, et retrouvez ce que le plan de DuckDB disait déjà : la lecture du Parquet avec son filtre poussé, la colonne du jour calculée, puis le regroupement. La ligne de plus, celle qui n'existe pas sur une machine, est l'Exchange : le brassage. Il est encadré par deux HashAggregate, et cet encadrement est ce qui rend l'opération tenable. Le premier agrège dans chaque partition et réduit 2,4 millions de points à quelques milliers de lignes ; seul ce résultat partiel traverse. Sans lui, tout traverserait.

```
cours 07 | agrégats : 11.1 s
cours 07 | agrégats : 47618, points : 2405366, somme des moyennes : 2737589.989
```

Onze secondes contre deux dixièmes, pour le même résultat, ou presque. Le « presque » mérite d'être regardé de près. Dans la ligne de commande DuckDB, depuis Projet_Boreal :

```
-- Les deux jeux d'agrégats, comparés ligne à ligne.
-- Attendu : 47 618 paires, 0 écart sur les points et les extrêmes, 6 moyennes différentes.
WITH une_machine AS (SELECT etiquette, jour, points, moyenne, minimum, maximum
                     FROM 'comparaison/duckdb/agregats.parquet'),
spark AS (SELECT TagName AS etiquette, jour, points, moyenne, minimum, maximum
          FROM 'comparaison/spark/agregats/*.parquet')
```

```

SELECT
  (SELECT COUNT(*) FROM une_machine JOIN spark USING (etiquette, jour)) AS paires,
  (SELECT COUNT(*) FROM une_machine JOIN spark USING (etiquette, jour)
   WHERE une_machine.points <> spark.points) AS points_differeents,
  (SELECT COUNT(*) FROM une_machine JOIN spark USING (etiquette, jour)
   WHERE une_machine.moyenne IS DISTINCT FROM spark.moyenne) AS moyennes_differeentes,
  (SELECT COUNT(*) FROM une_machine JOIN spark USING (etiquette, jour)
   WHERE une_machine.minimum IS DISTINCT FROM spark.minimum
    OR une_machine.maximum IS DISTINCT FROM spark.maximum) AS extremes_differeents;

```

Les comptes et les extrêmes sont identiques, et six moyennes sur 47 618 diffèrent. Regardez-les :

```

-- Les six moyennes qui ne tombent pas pareil.
-- Attendu : six lignes, toutes à 0,001 d'écart, dont UTL.CMP1.VIBRATION du 2025-04-25.
WITH une_machine AS (SELECT etiquette, jour, points, moyenne FROM 'comparaison/duckdb/agregats.parquet'),
     spark AS (SELECT TagName AS etiquette, jour, moyenne FROM 'comparaison/spark/agregats/*.parquet')
SELECT une_machine.etiquette, une_machine.jour, une_machine.points,
       une_machine.moyenne AS sur_une_machine, spark.moyenne AS sur_spark
FROM une_machine JOIN spark USING (etiquette, jour)
WHERE une_machine.moyenne IS DISTINCT FROM spark.moyenne
ORDER BY une_machine.etiquette;

```

Étiquette	Jour	Points	Une machine	Spark
LHR.CNV.VIBRATION	2025-01-03	40	0,503	0,504
LMM.CNV.PUISSANCE	2025-02-24	40	1,687	1,686
PEI.CAB.PUISSANCE	2025-12-09	40	89,124	89,123
UTL.CHF.TEMP_ALIMENTATION	2025-07-16	48	27,347	27,348
UTL.CMP1.VIBRATION	2025-04-25	92	2,027	2,028
UTL.SEC.POINT_ROSEE	2024-10-13	40	16,148	16,149

Aucun des deux ne s’est trompé. Une somme de nombres à virgule flottante dépend de l’ordre dans lequel on additionne, et les deux moteurs ne partitionnent pas le travail pareil. Sur ces six moyennes, le résultat exact tombe assez près d’un demi-millième pour que l’arrondi à trois décimales bascule d’un côté ou de l’autre.

À retenir — un contrôle d’égalité sur des flottants trouvera toujours un écart. La bonne forme est une tolérance : $ABS(a - b) \leq 0.001$, ou une comparaison sur les comptes et les extrêmes, qui sont exacts. Ce n’est pas une faiblesse de Spark, c’est l’arithmétique des flottants, et le Cours 11 en fera un contrat de données.

Point d'arrêt. Le même travail a tourné des deux côtés, et vous savez ce qu'il coûte. Si vous faites ce labo en deux séances, la première s'arrête ici : les étapes 6 à 8 ajoutent le pire cas, les trois pièges et la remise en état, et ne supposent rien de plus que ce dépôt.

Étape 6 — Dédoublonner la télémétrie, le pire cas pour Spark

Théorie : Theorie_07.md, section 4.

Le courtier MQTT livre « au moins une fois », donc l'archive contient 2 943 messages en double. Les écarter demande de numéroter les messages qui partagent un identifiant, ce qu'on fait par une fonction de fenêtrage, et une fenêtrage brasse tout.

Sur une machine, dans la ligne de commande DuckDB, depuis `Projet_Boreal` :

```
duckdb -cmd ".timer on" -c "COPY (SELECT jour, message.message_id AS message_id, message.sernr AS sernr, recu_le FROM
'plateforme/lac/bronze/telemetrie/*/*.parquet' QUALIFY ROW_NUMBER() OVER (PARTITION BY message.message_id ORDER BY recu_le) = 1)
TO 'comparaison/duckdb/telemetrie.parquet' (FORMAT parquet);"
```

Attendu : environ trois dixièmes de seconde. Le même en PySpark, dans l'Anaconda Prompt, depuis `01_de` :

```
notepad spark\telemetrie_dedoublonnee.py

# -*- coding: utf-8 -*-
"""telemetrie_dedoublonnee.py : un message par identifiant, en Spark (Cours 07)."""
import time

from pyspark.sql import SparkSession, functions as F
from pyspark.sql.window import Window

BRONZE = "plateforme/lac/bronze/telemetrie"
SORTIE = "comparaison/spark/telemetrie"

spark = SparkSession.builder.appName("boreal-telemetrie").getOrCreate()
spark.sparkContext.setLogLevel("WARN")

# La fenêtrage : un groupe par identifiant de message, ordonné par la réception.
fenetre = Window.partitionBy("message.message_id").orderBy("recu_le")

dedoublonne = (spark.read.parquet(BRONZE)
               .withColumn("rang", F.row_number().over(fenetre))
               .filter(F.col("rang") == 1)
               .select("jour",
                       F.col("message.message_id").alias("message_id"),
                       F.col("message.sernr").alias("sernr"),
                       "recu_le"))
```

```

debut = time.perf_counter()
dedoublonne.write.mode("overwrite").parquet(SORTIE)
print(f"cours 07 | dédoublonnage : {time.perf_counter() - debut:.1f} s")

# Contrôle : autant de messages gardés que d'identifiants distincts.
# Attendu : 201 542 messages pour 201 542 identifiants, 933 jours, 7 644 véhicules.
c = (spark.read.parquet(SORTIE)
     .agg(F.count("*").alias("messages"),
          F.countDistinct("message_id").alias("identifiants"),
          F.countDistinct("jour").alias("jours"),
          F.countDistinct("sernr").alias("vehicules"))
     .collect()[0])
print(f"cours 07 | gardés : {c['messages']}, identifiants : {c['identifiants']}, "
      f"jours : {c['jours']}, véhicules : {c['vehicules']}")

spark.stop()

```

Dans Ubuntu, depuis Projet_Boreal :

```

bash 01_de/spark/spark.sh 01_de/spark/telemetrie_dedoublonnee.py

cours 07 | dédoublonnage : 11.2 s
cours 07 | gardés : 201542, identifiants : 201542, jours : 933, véhicules : 7644

```

Mêmes 201 542 messages des deux côtés, sur les 204 485 reçus : les 2 943 doublons annoncés par le générateur sont écartés. La durée, elle, est le pire rapport du labo, une trentaine de fois celle d'une machine, parce qu'une fenêtre oblige tous les messages d'un même identifiant à se retrouver dans la même partition, et qu'il y en a 201 542.

Étape 7 — Les trois pièges, mesurés

Théorie : Theorie_07.md, section 7.

Dans l'Anaconda Prompt, depuis 01_de. Un dernier script, qui provoque exprès ce qu'on passe sa carrière à éviter :

```

notepad spark\pieges.py

# -*- coding: utf-8 -*-
"""pieges.py : Les trois pièges du traitement distribué, mesurés (Cours 07)."""
import time

from pyspark.sql import SparkSession, functions as F

```

```

BRONZE = "comparaison/spark/historien"

spark = SparkSession.builder.appName("boreal-pieges").getOrCreate()
spark.sparkContext.setLogLevel("WARN")
bronze = spark.read.parquet(BRONZE)

def agreger(source):
    """Le même agrégat que celui du cours, réduit à son compte de points."""
    return (source.filter(F.col("Quality") == 192)
            .groupBy("TagName", F.to_date("Timestamp").alias("jour"))
            .agg(F.count("*").alias("points")))

def chrono(nom, action):
    debut = time.perf_counter()
    action()
    print(f"cours 07 | {nom} : {time.perf_counter() - debut:.1f} s")

# Piège 1 : un brassage de plus, pour rien.
chrono("avec un repartition de trop", lambda: agreger(bronze.repartition(200, "TagName"))
        .write.mode("overwrite").parquet("comparaison/spark/piege_repartition"))
chrono("le même, sans repartition", lambda: agreger(bronze)
        .write.mode("overwrite").parquet("comparaison/spark/piege_sans"))

# Piège 2 : La partition déséquilibrée.
tailles = (bronze.repartition(58, "TagName")
          .withColumn("partition", F.spark_partition_id())
          .groupBy("partition").count()
          .orderBy(F.desc("count")).collect())
print(f"cours 07 | partitions non vides : {len(tailles)} sur 58 demandées")
print(f"cours 07 | la plus grosse : {tailles[0]['count']} lignes, "
      f"la plus petite : {tailles[-1]['count']}")

bavardes = bronze.groupBy("TagName").count().orderBy(F.desc("count")).limit(3).collect()
discretes = bronze.groupBy("TagName").count().orderBy("count").limit(3).collect()
print("cours 07 | les plus bavardes :", [(r["TagName"], r["count"]) for r in bavardes])
print("cours 07 | les plus discrètes :", [(r["TagName"], r["count"]) for r in discretes])

# Piège 3 : L'action qui ramène tout au pilote.
debut = time.perf_counter()

```

```

une = bronze.filter(F.col("TagName") == "UTL.CMP3.MARCHE").collect()
print(f"cours 07 | collect d'une étiquette : {len(une)} lignes en "
      f"{time.perf_counter() - debut:.1f} s")

debut = time.perf_counter()
tout = bronze.collect()
print(f"cours 07 | collect de tout : {len(tout)} lignes en {time.perf_counter() - debut:.1f} s")

spark.stop()

```

Dans Ubuntu, depuis Projet_Boreal :

```

bash 01_de/spark/spark.sh 01_de/spark/pieges.py

cours 07 | avec un repartition de trop : 18.0 s
cours 07 | le même, sans repartition : 8.8 s
cours 07 | partitions non vides : 37 sur 58 demandées
cours 07 | la plus grosse : 247879 lignes, la plus petite : 1833
cours 07 | les plus bavardes : [('ESS.BANC.TEMP_REFROIDISSEMENT', 242202), ('UTL.CHF.PUISSANCE', 188499), ('PEI.CAB.HUMIDITE', 71544)]
cours 07 | les plus discrètes : [('UTL.CMP3.MARCHE', 1833), ('LMM.CNV.MARCHE', 2378), ('LMN.ROB.MARCHE', 2501)]
cours 07 | collect d'une étiquette : 1833 lignes en 7.7 s
cours 07 | collect de tout : 2407796 lignes en 19.6 s

```

Trois leçons, dans l'ordre.

Le repartition avant l'agrégat double la durée, 18 secondes contre 8,8, et ne change rien au résultat. Il ajoute un brassage complet avant celui que l'agrégat allait faire de toute façon. C'est le geste « pour bien faire » le plus coûteux du métier.

Redistribuer sur l'étiquette donne 37 partitions non vides sur 58 demandées, parce que deux étiquettes tombent parfois dans la même case de hachage et que d'autres cases restent vides. Pire, les tailles n'ont aucun rapport : 247 879 lignes dans la plus grosse, 1 833 dans la plus petite. Le banc d'essai mesure la température de refroidissement de chaque véhicule terminé, 242 202 points en deux ans ; le compresseur d'appoint ne démarre presque jamais et n'écrit que son état, 1 833 fois. La durée du lot est celle de la plus grosse partition, et onze cœurs attendent le douzième.

Le collect d'une seule étiquette met 7,7 secondes pour 1 833 lignes : presque tout ce temps est le balayage du bronze, pas le transfert. Celui de tout le bronze ramène 2 407 796 lignes dans la mémoire du pilote en 19,6 secondes, et il passe, parce que le jeu est petit. Sur dix fois plus, le pilote tombe sur une erreur de mémoire qui ne nomme pas le coupable. Une action rend un résultat, pas un jeu de données.

Étape 8 — Conclure, remettre le projet d'aplomb, versionner

Théorie : Theorie_07.md, section 8.

Le tableau qui résume tout le labo, avec vos durées à vous :

Le travail	Une machine	Spark	Rapport
bronze de l'historien, 2 407 796 lignes	2,6 s	52 à 80 s	20 à 30 fois
agrégats quotidiens, 47 618 lignes	0,17 s	9 à 11 s	55 à 60 fois
dédoublonnage de la télémétrie, 201 542 messages	0,30 s	9 à 11 s	30 fois
ce que le lancement coûte en plus	aucun	20 à 30 s de démarrage du conteneur	

La conclusion du cours tient en une ligne : sur ce volume, le moteur d'une machine gagne de deux ordres de grandeur, et il gagne aussi en simplicité d'exploitation. Spark reste dans le dépôt parce qu'il faut savoir l'écrire, pas parce que Boréal en a besoin.

Remettez maintenant le projet dans l'état où le Cours 06 l'a laissé, sans quoi le Cours 08 ne retrouvera pas ses chiffres. Dans l'Anaconda Prompt, depuis Data_Domain :

```
python _univers/boreal/generer_boreal.py --projet Projet_Boreal --jusqua 2026-03
```

Puis, depuis 01_de, refaites ce qui en dérive :

```
python -m pipeline bronze  
  
INFO pipeline :   historien      550 087 lignes, source et bronze d'accord  
INFO pipeline :   télémétrie    204 485 lignes, source et bronze d'accord
```

L'historien est revenu à 182 jours. Effacez le dossier de comparaison, qui ne sert plus à rien et pèse une trentaine de mégaoctets :

```
rmdir /s /q ../comparaison
```

Enfin, versionnez le code de Spark. Un fichier d'abord, pour que Git ne remette pas les retours chariot de Windows dans le lanceur au prochain clone. Dans l'Anaconda Prompt, depuis 01_de :

```
notepad .gitattributes  
  
# Les scripts shell gardent des fins de ligne à la mode d'Unix, même sur un poste Windows :  
# bash refuse un retour chariot en fin de ligne.  
*.sh text eol=lf  
  
git status  
git add spark .gitattributes  
git commit -m "Cours 07 : les memes traitements en PySpark, et les trois pieges mesures"
```

Le dossier comparaison/ n'apparaît pas dans git status : il vivait dans Projet_Boreal, hors du dépôt, comme tout ce que le pipeline produit.

Récapitulatif

Ce que vous avez maintenant	Où
Le lanceur d'un conteneur à usage unique	01_de/spark/spark.sh
La conversion du bronze en PySpark	01_de/spark/convertir_historien.py
Les agrégats quotidiens en PySpark	01_de/spark/agregats_usine.py
Le dédoublonnage par fenêtre en PySpark	01_de/spark/telemetrie_dedoublonnee.py
Les trois pièges, provoqués et mesurés	01_de/spark/pieges.py
Le projet, revenu à l'état du Cours 06	Projet_Boreal/sources, historien de 182 jours

Le Cours 08 prend la suite, et change de sujet sans changer de données : d'où viennent ces points, ce qu'ils valent, et ce qu'il faut leur faire avant qu'un modèle ou un tableau de bord les lise.

Dépannage

Symptôme	Cause	Ce qu'on fait
docker: command not found dans Ubuntu	Docker Engine n'est pas installé dans cette distribution	étape 5 du Data_Domain/GUIDE_INSTALLATION.md
docker: permission denied while trying to connect	votre compte Ubuntu n'est pas dans le groupe docker	sudo usermod -aG docker \$USER, puis rouvrir Ubuntu
java.io.FileNotFoundException: File sources/usine/historien/*.csv does not exist	Spark sonde le chemin littéral avant d'étendre le joker	rien : le compte final est juste, le message est sans conséquence
chmod: changing permissions of '/projet/comparaison/spark/historien': Operation not permitted	le conteneur tourne sous un autre utilisateur que le propriétaire du montage	garder --user root dans spark.sh
java.io.IOException: Invalid UID, could not determine effective user	--user désigne un identifiant qui n'existe pas dans l'image	--user root, le seul compte sûr de cette image

Symptôme	Cause	Ce qu'on fait
set: -: invalid option ou \$\r': command not found	spark.sh a été enregistré avec les fins de ligne de Windows	sed -i 's/\r\$//' 01_de/spark/spark.sh dans Ubuntu, et le .gitattributes de l'étape 8
py4j.protocol.Py4JJavaError sur Path does not exist: file:/projet/...	le script est lancé depuis un autre dossier que Projet_Boreal	cd vers Projet_Boreal dans Ubuntu, puis relancer : les chemins du script sont relatifs au projet
AnalysisException: [PATH_ALREADY_EXISTS]	une écriture sans mode("overwrite")	ajouter le mode, comme dans les scripts du labo
Les agrégats de Spark ne comptent pas comme ceux d'une machine	le bronze des deux côtés ne vient pas de la même génération	refaire l'étape 1 en entier, puis l'étape 3
Spark met plus de trois minutes sur la conversion	le conteneur partage la machine avec la pile Airflow du Cours 06	docker compose down dans plateforme, puis relancer
python -m pipeline bronze rend 550 087 lignes d'historien à l'étape 1	les sources n'ont pas été régénérées avec --historien complet	refaire la commande de l'étape 1, depuis Data_Domain

Annexe — Sur Databricks

Pour qui veut voir le même code sur la plateforme que l'industrie nomme le plus souvent quand elle dit « Spark ». Les faits ont été vérifiés dans la documentation de Databricks en septembre 2026.

Databricks Free Edition suffit pour cette annexe, et ses limites méritent d'être dites avant de commencer, parce qu'elles décident de ce qu'on peut montrer :

La limite	Ce qu'elle change
calcul serverless seulement, aucune configuration de calcul personnalisée	le dimensionnement d'une grappe, le nombre d'exécuteurs et leur mémoire ne se règlent pas, donc ne se montrent pas
R et Scala non pris en charge	l'annexe est en PySpark, comme le labo
un seul entrepôt SQL, en taille 2X-Small	les mesures de durée n'ont rien à voir avec celles du labo, et ne se comparent pas
cinq tâches de travail simultanées par compte	les essais se font un à la fois
quota dépassé, le calcul s'arrête pour le reste de la journée	les données et les réglages restent intacts ; le travail reprend le lendemain
aucun usage commercial	pour apprendre, pas pour un mandat

Le code du labo tourne tel quel, à trois détails près. La session existe déjà : dans un carnet Databricks, la variable `spark` est fournie, et les lignes `SparkSession.builder` ne servent plus. Les chemins changent : les fichiers vivent dans un volume Unity Catalog, `/Volumes/<catalogue>/<schéma>/<volume>/...`, et non dans un dossier du poste. Et l'écriture se fait en Delta plutôt qu'en Parquet nu, parce que c'est le format par défaut de la plateforme.

```
# Dans un carnet Databricks : « spark » existe déjà.
from pyspark.sql import functions as F, types as T

SCHEMA = T.StructType([
    T.StructField("TagName", T.StringType()),
    T.StructField("Timestamp", T.TimestampType()),
    T.StructField("Value", T.DoubleType()),
    T.StructField("Quality", T.IntegerType()),
])

historien = (spark.read
    .option("header", True)
    .option("timestampFormat", "yyyy-MM-dd HH:mm:ss")
    .schema(SCHEMA)
    .csv("/Volumes/<catalogue>/bronze/livraisons/usine/historien/")
    .withColumn("jour", F.to_date(F.regexp_extract(
        F.input_file_name(), r"historien_(\d{4}-\d{2}-\d{2})", 1))))

(historien.write.mode("overwrite").partitionBy("jour")
    .saveAsTable("bronze.usine_historien"))
```

Ce que la plateforme apporte par-dessus le moteur, et que le labo ne peut pas montrer en local : le catalogue et ses permissions, l'historique des versions d'une table Delta, le suivi des traitements, et le fait que la donnée vive déjà à côté du calcul. Ce que le labo montre et que la plateforme cache : ce que coûte un brassage, et combien de partitions un travail crée. Les deux savoirs se complètent, et c'est le second qui fait diagnostiquer une lenteur.

Le format Delta, le voyage dans le temps et le catalogue reviennent au Cours 10, qui les traite pour de bon.